

C++ Object-Oriented Design & SOLID Principles Study Guide

Table of Contents

1. [Object-Oriented Design Fundamentals](#)
 2. [Object Relationships](#)
 3. [SOLID Design Principles](#)
 4. [UML Diagrams](#)
 5. [Practice Questions with Answers](#)
-

1. Object-Oriented Design Fundamentals {#oo-fundamentals}

1.1 Abstraction

Definition: A model of a real-world object that includes only relevant details for a specific context.

Key Points:

- Objects don't represent 100% of real-world entities
- Only model attributes and behaviors relevant to the context
- Hide unnecessary complexity

Example: An `Airplane` class in a flight simulator vs. booking system

- Flight simulator: speed, altitude, fuel level
- Booking system: seat map, availability

1.2 Encapsulation

Definition: Hiding internal details and exposing only necessary interfaces.

Key Points:

- Make attributes private
- Provide public methods for controlled access
- Protect internal state from direct modification

Example:

cpp

```
#include <iostream>
```

```
#include <string>
```

```
enum class SpotSize { SMALL, MEDIUM, LARGE };
```

```
class ParkingSpot {
```

```
private:
```

```
    int spotId;          // Private - internal identifier
```

```
    bool isOccupied;      // Private - internal state
```

```
    SpotSize size;        // Private - internal property
```

```
public:
```

```
    // Constructor
```

```
    ParkingSpot(int id, SpotSize spotSize)
```

```
        : spotId(id), isOccupied(false), size(spotSize) {}
```

```
    // Public - controlled access
```

```
    bool isAvailable() const {
```

```
        return !isOccupied;
```

```
    }
```

```
    // Public - controlled modification
```

```
    void occupy() {
```

```
        if (!isOccupied) {
```

```
            isOccupied = true;
```

```
        }
```

```
    }
```

```
    void vacate() {
```

```
        isOccupied = false;
```

```
    }
```

```
    SpotSize getSize() const {
```

```
        return size;  
    }  
};
```

1.3 Inheritance

Definition: Building new classes based on existing ones to reuse code.

Key Points:

- Subclasses inherit fields and methods from superclass
- Enables code reuse
- Establishes "is-a" relationship

Example:

```
cpp
```

```
#include <string>
#include <iostream>

class Vehicle {
protected:
    std::string licensePlate;
    std::string color;

public:
    Vehicle(const std::string& plate, const std::string& vehicleColor)
        : licensePlate(plate), color(vehicleColor) {}

    virtual ~Vehicle() = default; // Virtual destructor for proper cleanup

    virtual void park() {
        std::cout << "Parking vehicle with license plate: " << licensePlate << std::endl;
    }

    std::string getLicensePlate() const {
        return licensePlate;
    }
};

class Car : public Vehicle {
private:
    int numberOfDoors;

public:
    Car(const std::string& plate, const std::string& vehicleColor, int doors)
        : Vehicle(plate, vehicleColor), numberOfDoors(doors) {}

    void park() override {
        std::cout << "Parking car with " << numberOfDoors << " doors" << std::endl;
    }
};
```

```
}  
  
int getDoors() const {  
    return numberOfDoors;  
}  
};
```

1.4 Polymorphism

Definition: Ability to treat objects of different types uniformly through a common interface.

Key Points:

- Same method call, different behaviors
- Enables flexible, extensible code
- Runtime method binding through virtual functions

Example:

cpp

```
#include <memory>

void parkVehicle(std::unique_ptr<Vehicle>& vehicle) {
    vehicle->park(); // Calls appropriate method based on actual type
}

// Usage example
int main() {
    std::unique_ptr<Vehicle> car = std::make_unique<Car>("ABC123", "Red", 4);
    parkVehicle(car); // Calls Car::park()

    return 0;
}
```

2. Object Relationships {#object-relationships}

2.1 Association

- **Definition:** "Using" relationship between independent objects
- **Characteristics:** Objects have their own lifecycle, no ownership
- **Example:** Doctor-Patient relationship

2.2 Aggregation (Has-A)

- **Definition:** Specialized association with ownership but independent lifecycle
- **Characteristics:** Parent can exist without child and vice versa
- **Example:** Department-Employee relationship

2.3 Composition (Part-Of)

- **Definition:** Strong ownership where child cannot exist without parent

- **Characteristics:** Parent controls child's lifecycle
- **Example:** House-Room relationship

2.4 Inheritance (Is-A)

- **Definition:** Subclass inherits properties from superclass
 - **Characteristics:** Establishes hierarchical relationship
 - **Example:** Car-Vehicle relationship
-

3. SOLID Design Principles {#solid-principles}

3.1 Single Responsibility Principle (SRP)

Definition: A class should have only one reason to change.

Example Problem:

```
cpp
// BAD: Multiple responsibilities
class ParkingLot {
public:
    void parkVehicle(Vehicle* v) { /* parking logic */}
    double calculateFee(Vehicle* v) { /* fee calculation */}
    void generateReport() { /* reporting logic */}
    void processPayment() { /* payment logic */}
};
```

Solution:

```
cpp
```


// GOOD: Separate responsibilities

```
class ParkingLot {  
public:  
    void parkVehicle(Vehicle* v) {  
        /* only parking logic */  
    }  
};
```

```
class FeeCalculator {  
public:  
    double calculateFee(Vehicle* v) {  
        /* only fee calculation */  
    }  
};
```

```
class ReportGenerator {  
public:  
    void generateReport() {  
        /* only reporting logic */  
    }  
};
```

```
class PaymentProcessor {  
public:  
    void processPayment() {  
        /* only payment logic */  
    }  
};
```

3.2 Open/Closed Principle (OCP)

Definition: Classes should be open for extension but closed for modification.

Example:

cpp

// Abstract base for extension

```
class Vehicle {  
public:  
    virtual ~Vehicle() = default;  
    virtual double calculateParkingFee(int hours) = 0;  
};
```

// Concrete implementations

```
class Car : public Vehicle {  
public:  
    double calculateParkingFee(int hours) override {  
        return hours * 2.0; // $2 per hour  
    }  
};
```

```
class Bus : public Vehicle {  
public:  
    double calculateParkingFee(int hours) override {  
        return hours * 5.0; // $5 per hour  
    }  
};
```

// Adding new vehicle type without modifying existing code

```
class Motorcycle : public Vehicle {  
public:  
    double calculateParkingFee(int hours) override {  
        return hours * 1.0; // $1 per hour  
    }  
};
```

3.3 Liskov Substitution Principle (LSP)

Definition: Objects of a superclass should be replaceable with objects of subclasses without breaking functionality.

Guidelines:

- Subclass methods should accept same or more general parameters
- Subclass methods should return same or more specific types
- Don't throw new exceptions in subclass methods
- Don't strengthen preconditions or weaken postconditions

Example:

cpp

```

class Rectangle {
protected:
    int width, height;

public:
    Rectangle(int w, int h) : width(w), height(h) {}

    virtual void setWidth(int w) { width = w; }
    virtual void setHeight(int h) { height = h; }

    virtual int getWidth() const { return width; }
    virtual int getHeight() const { return height; }

    virtual int area() const { return width * height; }
};

// LSP compliant - Square behaves like Rectangle
class Square : public Rectangle {
public:
    Square(int side) : Rectangle(side, side) {}

    void setWidth(int w) override {
        width = height = w;
    }

    void setHeight(int h) override {
        width = height = h;
    }
};

```

3.4 Interface Segregation Principle (ISP)

Definition: Clients shouldn't be forced to depend on methods they don't use.

Example:

cpp

// BAD: Monolithic interface

```
class ParkingLotInterface {  
public:  
    virtual void parkVehicle(Vehicle* v) = 0;  
    virtual void processPayment(Payment* p) = 0;  
    virtual void generateReport() = 0;  
    virtual void updateDisplayBoard() = 0;  
};
```

// GOOD: Segregated interfaces

```
class ParkingOperations {  
public:  
    virtual ~ParkingOperations() = default;  
    virtual void parkVehicle(Vehicle* v) = 0;  
    virtual void unparkVehicle(const std::string& licensePlate) = 0;  
};
```

```
class PaymentOperations {  
public:  
    virtual ~PaymentOperations() = default;  
    virtual void processPayment(Payment* p) = 0;  
};
```

```
class ReportingOperations {  
public:  
    virtual ~ReportingOperations() = default;  
    virtual void generateReport() = 0;  
};
```

// Implementation classes use only needed interfaces

```
class ParkingLot : public ParkingOperations {  
public:  
    void parkVehicle(Vehicle* v) override {
```

```
    /* implementation */  
}  
  
void unparkVehicle(const std::string& licensePlate) override {  
    /* implementation */  
}  
};
```

3.5 Dependency Inversion Principle (DIP)

Definition: High-level modules shouldn't depend on low-level modules. Both should depend on abstractions.

Example:

cpp

// BAD: Direct dependency

```
class MySQLDatabase {  
public:  
    void save(const std::string& data) {  
        // MySQL specific implementation  
    }  
};
```

```
class ParkingLot {
```

```
private:
```

```
    MySQLDatabase database; // Depends on concrete class
```

```
public:
```

```
    void saveVehicleData(const std::string& data) {  
        database.save(data);  
    }  
};
```

// GOOD: Depend on abstraction

```
class DatabaseInterface {
```

```
public:
```

```
    virtual ~DatabaseInterface() = default;  
    virtual void save(const std::string& data) = 0;  
};
```

```
class MySQLDatabase : public DatabaseInterface {
```

```
public:
```

```
    void save(const std::string& data) override {  
        // MySQL specific implementation  
    }  
};
```

```
class ParkingLot {
```


private:

```
std::unique_ptr<DatabaseInterface> database; // Depends on abstraction
```

public:

```
ParkingLot(std::unique_ptr<DatabaseInterface> db)
```

```
: database(std::move(db)) {}
```

```
void saveVehicleData(const std::string& data) {
```

```
    database->save(data);
```

```
}
```

```
};
```

4. UML Diagrams {#uml-diagrams}

4.1 Class Diagram Elements

- **Class:** Rectangle with class name, attributes, methods
- **Association:** Simple line (1-to-1, 1-to-many)
- **Aggregation:** Line with empty diamond
- **Composition:** Line with filled diamond
- **Inheritance:** Line with empty triangle arrow

4.2 Example Class Diagram

```

Vehicle
-----
- licensePlate: string
- color: string
-----
+ park(): void
+ calculateFee(hours: int): double
  ^
  |
-----
|   |
Car  Bus
---  ---
- doors - capacity

```

5. Practice Questions with Answers {#practice-questions}

Practice Question 1: Abstraction

Question: Describe how abstraction would be applied to a `Vehicle` class in a parking lot system.

Answer: The `Vehicle` class would be an abstraction that includes only parking-relevant attributes and behaviors:

- **Included:** licensePlate, size, parkingSpotRequired, calculateFee()
- **Excluded:** engine specifications, interior details, manufacturing date
- **Reasoning:** The parking system only needs to know vehicle identification, space requirements, and fee calculation - not mechanical details.

Practice Question 2: Encapsulation

Question: Design a `ParkingSpot` class with proper encapsulation.

Answer:

cpp

```
class ParkingSpot {
private:
    // Private attributes - encapsulated
    int spotId;
    bool isOccupied;
    SpotSize size;
    Vehicle* currentVehicle;

public:
    // Constructor
    ParkingSpot(int id, SpotSize spotSize)
        : spotId(id), isOccupied(false), size(spotSize), currentVehicle(nullptr) {}

    // Public methods - controlled access
    bool isAvailable() const {
        return !isOccupied;
    }

    void occupy(Vehicle* vehicle) {
        if (!isOccupied && vehicle != nullptr) {
            isOccupied = true;
            currentVehicle = vehicle;
        }
    }

    void vacate() {
        isOccupied = false;
        currentVehicle = nullptr;
    }

    SpotSize getSize() const {
        return size;
    }
}
```

```
Vehicle* getCurrentVehicle() const {  
    return currentVehicle;  
}  
};
```

Practice Question 3: Polymorphism

Question: Explain how polymorphism enables flexible fee calculation.

Answer:

cpp

```

class FeeCalculator {
public:
    double calculateFee(Vehicle* vehicle, int hours) {
        // Polymorphic call - actual method depends on vehicle type
        return vehicle->calculateParkingFee(hours);
    }
};

// Different implementations
class Car : public Vehicle {
public:
    double calculateParkingFee(int hours) override {
        return hours * 2.0;
    }
};

class Bus : public Vehicle {
public:
    double calculateParkingFee(int hours) override {
        return hours * 5.0;
    }
};

// Usage
int main() {
    FeeCalculator calculator;

    std::unique_ptr<Vehicle> car = std::make_unique<Car>("ABC123", "Red", 4);
    std::unique_ptr<Vehicle> bus = std::make_unique<Bus>("XYZ789", "Blue", 50);

    double carFee = calculator.calculateFee(car.get(), 2); // $4.0
    double busFee = calculator.calculateFee(bus.get(), 2); // $10.0

```

```
    return 0;  
}
```

Practice Question 4: Object Relationships

Question: Identify relationships between ParkingLot, ParkingSpot, and Vehicle.

Answer:

```
cpp
```

```

#include <vector>
#include <memory>

class ParkingLot {
private:
    std::vector<std::unique_ptr<ParkingSpot>> spots; // Composition
    std::vector<Vehicle*> parkedVehicles;          // Association

public:
    // ParkingLot owns ParkingSpots - Composition
    ParkingLot(int numSpots) {
        for (int i = 0; i < numSpots; ++i) {
            spots.push_back(std::make_unique<ParkingSpot>(i, SpotSize::MEDIUM));
        }
    }

    // ParkingLot manages Vehicles temporarily - Association
    bool parkVehicle(Vehicle* vehicle) {
        for (auto& spot : spots) {
            if (spot->isAvailable()) {
                spot->occupy(vehicle);
                parkedVehicles.push_back(vehicle);
                return true;
            }
        }
        return false;
    }
};

```

Relationships:

1. **ParkingLot** ↔ **ParkingSpot**: Composition (ParkingLot owns ParkingSpots)

2. **ParkingLot** ↔ **Vehicle**: Association (temporary management)

3. **ParkingSpot** ↔ **Vehicle**: Association (temporary occupancy)

Practice Question 5: Single Responsibility Principle

Question: Refactor a monolithic ParkingLot class following SRP.

Answer:

cpp

// Original - violates SRP

```
class ParkingLot {
public:
    void parkVehicle(Vehicle* v) { /* parking */}
    double calculateFee(Vehicle* v) { /* fee calculation */}
    void generateReport() { /* reporting */}
    void processPayment() { /* payment */}
};
```

// Refactored - follows SRP

```
class ParkingLot {
public:
    void parkVehicle(Vehicle* v) {
        /* only parking logic */
    }

    void unparkVehicle(const std::string& licensePlate) {
        /* only unparking logic */
    }
};
```

```
class FeeCalculator {
public:
    double calculateFee(Vehicle* v, int hours) {
        return v->calculateParkingFee(hours);
    }
};
```

```
class ReportGenerator {
public:
    void generateDailyReport() {
        /* only daily reporting logic */
    }
};
```

```
void generateMonthlyReport() {  
    /* only monthly reporting logic */  
}  
};  
  
class PaymentProcessor {  
public:  
    void processPayment(double amount) {  
        /* only payment processing logic */  
    }  
};
```

Practice Question 6: Open/Closed Principle

Question: Design a system that supports new vehicle types without modifying existing code.

Answer:

cpp

// Abstract base - closed for modification

```
class Vehicle {  
protected:  
    std::string licensePlate;  
  
public:  
    Vehicle(const std::string& plate) : licensePlate(plate) {}  
    virtual ~Vehicle() = default;  
  
    virtual SpotSize getRequiredSpotSize() const = 0;  
    virtual double calculateParkingFee(int hours) const = 0;  
  
    std::string getLicensePlate() const {  
        return licensePlate;  
    }  
};
```

// Concrete implementations - open for extension

```
class Car : public Vehicle {  
public:  
    Car(const std::string& plate) : Vehicle(plate) {}  
  
    SpotSize getRequiredSpotSize() const override {  
        return SpotSize::MEDIUM;  
    }  
  
    double calculateParkingFee(int hours) const override {  
        return hours * 2.0;  
    }  
};
```

```
class ElectricVehicle : public Vehicle {  
public:
```

```

ElectricVehicle(const std::string& plate) : Vehicle(plate) {}

SpotSize getRequiredSpotSize() const override {
    return SpotSize::MEDIUM;
}

double calculateParkingFee(int hours) const override {
    return hours * 1.5; // Discounted rate
}
};

// New vehicle type added without modifying existing code
class Truck : public Vehicle {
public:
    Truck(const std::string& plate) : Vehicle(plate) {}

    SpotSize getRequiredSpotSize() const override {
        return SpotSize::LARGE;
    }

    double calculateParkingFee(int hours) const override {
        return hours * 8.0;
    }
};

```

Practice Question 7: Liskov Substitution Principle

Question: Provide an example of LSP violation and its fix.

Answer: Violation Example:

cpp

```
#include <stdexcept>

class Vehicle {
public:
    virtual void park() {
        std::cout << "Vehicle parked in regular spot" << std::endl;
    }
};

class DisabledVehicle : public Vehicle {
public:
    void park() override {
        throw std::runtime_error("Cannot park - needs special parking");
    }
};

// This violates LSP - substituting DisabledVehicle breaks functionality
```

Fix:

cpp

// Abstract base class

```
class Vehicle {  
public:  
    virtual ~Vehicle() = default;  
    virtual void park() = 0;  
    virtual bool needsSpecialParking() const = 0;  
};
```

```
class RegularVehicle : public Vehicle {  
public:  
    void park() override {  
        std::cout << "Vehicle parked in regular spot" << std::endl;  
    }  
  
    bool needsSpecialParking() const override {  
        return false;  
    }  
};
```

```
class DisabledVehicle : public Vehicle {  
public:  
    void park() override {  
        std::cout << "Vehicle parked in disabled-accessible spot" << std::endl;  
    }  
  
    bool needsSpecialParking() const override {  
        return true;  
    }  
};
```

// Now both can be used interchangeably

```
void parkVehicle(Vehicle* vehicle) {
```

```
vehicle->park(); // Works for both types  
}
```

Practice Question 8: Interface Segregation Principle

Question: Apply ISP to ParkingLot interfaces.

Answer:

```
cpp
```


// Segregated interfaces

```
class ParkingOperations {  
public:  
    virtual ~ParkingOperations() = default;  
    virtual void parkVehicle(Vehicle* vehicle) = 0;  
    virtual void unparkVehicle(const std::string& licensePlate) = 0;  
};
```

```
class PaymentOperations {  
public:  
    virtual ~PaymentOperations() = default;  
    virtual void processPayment(double amount) = 0;  
};
```

```
class ReportingOperations {  
public:  
    virtual ~ReportingOperations() = default;  
    virtual void generateDailyReport() = 0;  
    virtual void generateMonthlyReport() = 0;  
};
```

// Classes implement only needed interfaces

```
class ParkingLot : public ParkingOperations {  
private:  
    std::vector<std::unique_ptr<ParkingSpot>> spots;  
  
public:  
    void parkVehicle(Vehicle* vehicle) override {  
        /* implementation */  
    }  
  
    void unparkVehicle(const std::string& licensePlate) override {  
        /* implementation */  
    }  
};
```

```

    }
};

class ReportGenerator : public ReportingOperations {
public:
    void generateDailyReport() override {
        /* implementation */
    }

    void generateMonthlyReport() override {
        /* implementation */
    }
};

class PaymentProcessor : public PaymentOperations {
public:
    void processPayment(double amount) override {
        /* implementation */
    }
};

```

Key C++ Specific Considerations

Memory Management

- Use smart pointers (`std::unique_ptr`, `std::shared_ptr`) for automatic memory management
- Implement proper destructors, especially virtual destructors for base classes
- Follow RAII (Resource Acquisition Is Initialization) principles

Virtual Functions

- Use `virtual` keyword for polymorphic behavior

- Use `override` keyword in derived classes for safety
- Pure virtual functions (`= 0`) for abstract base classes

Const Correctness

- Mark methods that don't modify object state as `const`
 - Use `const` references for parameters when appropriate
 - Helps with optimization and prevents accidental modifications
-

Key Takeaways for Assignment Success

1. **Understand the Context:** Always consider what the parking lot system needs to do
2. **Apply Principles Correctly:** Don't just memorize - understand why each principle matters
3. **Use UML Properly:** Show relationships clearly with correct notation
4. **Think Extensibility:** Design for future changes (new vehicle types, fee structures)
5. **Justify Your Decisions:** Explain why you chose specific designs
6. **C++ Best Practices:** Use smart pointers, virtual destructors, and const correctness

Final Tips

- **Practice Drawing UML:** Get comfortable with class diagrams and relationship notation
 - **Think Real-World:** Consider how the system would actually work in practice
 - **Review SOLID:** These principles are fundamental to good design
 - **Start Simple:** Begin with basic relationships, then add complexity
 - **Test Your Design:** Ask "What if we add a new vehicle type?" to validate extensibility
 - **Memory Safety:** Always consider object lifetimes and use appropriate smart pointers
 - **Virtual Destructors:** Essential for proper cleanup in inheritance hierarchies
-

Parking Lot System - Assignment Practice Questions & Answers

Part 1: Object-Oriented Design Fundamentals (50 Marks)

Question 1: Abstraction (5 Marks)

Question: Describe an example of an object relationship you would propose based on the Parking Lot Management System to model the object-oriented principle of Abstraction.

Model Answer: The **Vehicle** class serves as an abstraction for the parking lot system. In the real world, vehicles have many attributes (engine type, manufacturer, interior design, etc.), but for parking lot management, we only need:

Relevant Attributes:

- `licensePlate` (for identification)
- `vehicleType` (for spot allocation)
- `entryTime` (for fee calculation)
- `spotSize` (for space requirements)

Relevant Methods:

- `calculateParkingFee(hours)`
- `getRequiredSpotSize()`

Irrelevant Details Omitted:

- Engine specifications, fuel type, interior features, manufacturing details

This abstraction allows the parking system to work with any vehicle type while ignoring unnecessary complexity.

Question 2: Encapsulation (10 Marks)

Part A: Private Attributes (5 Marks)

Question: Propose 3 examples of attributes you would make private within the ParkingSpot class.

Model Answer:

```
cpp

class ParkingSpot {
private:
    int spotId;           // Internal identifier
    bool isOccupied;      // Internal state tracking
    SpotSize size;        // Internal property
    std::unique_ptr<Vehicle> currentVehicle; // Internal reference

public:
    // Constructor
    ParkingSpot(int id, SpotSize spotSize)
        : spotId(id), isOccupied(false), size(spotSize), currentVehicle(nullptr) {}
};
```

Reasons for Privacy:

1. **spotId:** Internal identifier shouldn't be changed by external classes
2. **isOccupied:** Direct modification could cause inconsistent state
3. **size:** Fixed property that shouldn't be modified after creation

Part B: Public Methods (5 Marks)

Question: Propose 2 examples of public functions and explain how they protect internal state.

Model Answer:

cpp

```
class ParkingSpot {
private:
    int spotId;
    bool isOccupied;
    SpotSize size;
    Vehicle* currentVehicle;

public:
    // Public methods for controlled access
    bool isAvailable() const {
        return !isOccupied;
    }

    bool assignVehicle(Vehicle* vehicle) {
        if (!isOccupied && vehicle && vehicle->getRequiredSpotSize() == this->size) {
            this->isOccupied = true;
            this->currentVehicle = vehicle;
            return true;
        }
        return false;
    }

    void releaseVehicle() {
        this->isOccupied = false;
        this->currentVehicle = nullptr;
    }
};
```

Protection Mechanisms:

1. **isAvailable()**: Provides read-only access to occupancy status without exposing internal boolean
 2. **assignVehicle()**: Validates conditions before modifying state, preventing invalid assignments
-

Question 3: Inheritance and Polymorphism (25 Marks)

Part A: Unique Subclass Attributes (4 Marks)

Question: What specific attributes would each subclass uniquely define?

Model Answer:

cpp

```
class Car : public Vehicle {
private:
    int numberOfDoors;
    std::string fuelType;
    bool hasAirConditioning;

public:
    Car(const std::string& plate, int doors, const std::string& fuel, bool ac)
        : Vehicle(plate), numberOfDoors(doors), fuelType(fuel), hasAirConditioning(ac) {}
};
```

```
class Motorcycle : public Vehicle {
private:
    int engineCapacity;
    bool hasSidecar;
    std::string motorcycleType;

public:
    Motorcycle(const std::string& plate, int capacity, bool sidecar, const std::string& type)
        : Vehicle(plate), engineCapacity(capacity), hasSidecar(sidecar), motorcycleType(type) {}
};
```

```
class Bus : public Vehicle {
private:
    int passengerCapacity;
    int numberOfAxles;
    bool isSchoolBus;

public:
    Bus(const std::string& plate, int capacity, int axles, bool school)
        : Vehicle(plate), passengerCapacity(capacity), numberOfAxles(axles), isSchoolBus(school) {}
};
```


Part B: Shared Attributes (4 Marks)

Question: Give examples of two attributes shared among Vehicle and subclasses.

Model Answer:

```
cpp

#include <chrono>
#include <string>

class Vehicle {
protected:
    std::string licensePlate;           // Common to all vehicles
    std::string color;                 // Common to all vehicles
    std::chrono::system_clock::time_point entryTime; // Common for parking tracking

public:
    Vehicle(const std::string& plate) : licensePlate(plate) {}
    virtual ~Vehicle() = default;

    std::string getLicensePlate() const { return licensePlate; }
    void setColor(const std::string& vehicleColor) { color = vehicleColor; }
    void setEntryTime(const std::chrono::system_clock::time_point& time) { entryTime = time; }
};
```

Part C: Overridden Methods (4 Marks)

Question: Provide examples of two Vehicle methods that would be overridden by subclasses.

Model Answer:

```
cpp
```

```
class Vehicle {
public:
    virtual ~Vehicle() = default;
    virtual double calculateParkingFee(int hours) const = 0;
    virtual SpotSize getRequiredSpotSize() const = 0;
};
```

// Subclass implementations

```
class Car : public Vehicle {
public:
    double calculateParkingFee(int hours) const override {
        return hours * 2.0; // $2 per hour for cars
    }

    SpotSize getRequiredSpotSize() const override {
        return SpotSize::MEDIUM;
    }
};
```

```
class Bus : public Vehicle {
public:
    double calculateParkingFee(int hours) const override {
        return hours * 5.0; // $5 per hour for buses
    }

    SpotSize getRequiredSpotSize() const override {
        return SpotSize::LARGE;
    }
};
```

```
class Motorcycle : public Vehicle {
public:
    double calculateParkingFee(int hours) const override {
```

```
        return hours * 1.0; // $1 per hour for motorcycles
    }

    SpotSize getRequiredSpotSize() const override {
        return SpotSize::SMALL;
    }
};
```

Part D: Polymorphism in parkVehicle (4 Marks)

Question: Explain how polymorphism allows parkVehicle function to handle different vehicle types.

Model Answer:

cpp

```

#include <vector>
#include <memory>

class ParkingLot {
private:
    std::vector<std::unique_ptr<ParkingSpot>> spots;

    ParkingSpot* findAvailableSpot(SpotSize requiredSize) {
        for (auto& spot : spots) {
            if (spot->isAvailable() && spot->getSize() == requiredSize) {
                return spot.get();
            }
        }
        return nullptr;
    }

public:
    bool parkVehicle(Vehicle* vehicle) {
        // Polymorphic call - method resolution at runtime
        SpotSize requiredSize = vehicle->getRequiredSpotSize();

        ParkingSpot* availableSpot = findAvailableSpot(requiredSize);

        if (availableSpot != nullptr) {
            return availableSpot->assignVehicle(vehicle);
        }
        return false;
    }
};

```

Polymorphism Benefits:

- Single method handles all vehicle types
- No explicit type checking needed
- New vehicle types can be added without modifying this method
- Runtime method binding calls appropriate subclass implementation

Part E: Polymorphism in Fee Calculation (4 Marks)

Question: Describe how the system leverages polymorphism for fee calculation.

Model Answer:

```
cpp

#include <chrono>

class FeeCalculator {
private:
    int calculateHours(const std::chrono::system_clock::time_point& entryTime,
                      const std::chrono::system_clock::time_point& exitTime) const {
        auto duration = std::chrono::duration_cast<std::chrono::hours>(exitTime - entryTime);
        return static_cast<int>(duration.count());
    }

public:
    double calculateFee(Vehicle* vehicle, const std::chrono::system_clock::time_point& exitTime) {
        int hours = calculateHours(vehicle->getEntryTime(), exitTime);

        // Polymorphic call - each vehicle type has different fee structure
        return vehicle->calculateParkingFee(hours);
    }
};
```

Benefits:

- Same calculation logic works for all vehicle types
- Each vehicle type implements its own fee structure
- Easy to add new vehicle types with different pricing
- No conditional statements needed

Part F: Object Relationships (10 Marks)

Question: Describe relationships among ParkingLot, ParkingSpot, and Vehicle classes.

Model Answer:**1. ParkingLot ↔ ParkingSpot: Composition**

- ParkingLot owns ParkingSpots
- ParkingSpots cannot exist without ParkingLot
- When ParkingLot is destroyed, all ParkingSpots are destroyed
- Relationship: 1 ParkingLot has many ParkingSpots

2. ParkingLot ↔ Vehicle: Association

- ParkingLot manages Vehicles temporarily
- Vehicles exist independently of ParkingLot
- Weak "uses" relationship during parking duration
- Relationship: 1 ParkingLot can manage many Vehicles

3. ParkingSpot ↔ Vehicle: Association

- ParkingSpot can be occupied by Vehicle
- Both exist independently

- Temporary relationship during parking
- Relationship: 1 ParkingSpot can accommodate 1 Vehicle at a time

C++ Implementation Example:

cpp

```

class ParkingLot {
private:
    std::vector<std::unique_ptr<ParkingSpot>> spots; // Composition
    std::vector<Vehicle*> parkedVehicles;          // Association

public:
    ParkingLot(int numSpots) {
        for (int i = 0; i < numSpots; ++i) {
            spots.push_back(std::make_unique<ParkingSpot>(i, SpotSize::MEDIUM));
        }
    }

    bool parkVehicle(Vehicle* vehicle) {
        // Association - temporary management
        SpotSize requiredSize = vehicle->getRequiredSpotSize();

        for (auto& spot : spots) {
            if (spot->isAvailable() && spot->getSize() == requiredSize) {
                if (spot->assignVehicle(vehicle)) {
                    parkedVehicles.push_back(vehicle);
                    return true;
                }
            }
        }
        return false;
    }
};

```

Part G: UML Class Diagram (5 Marks)

Question: Draw a UML class relationship diagram.

Model Answer:



Legend:

-  = Composition (ParkingLot owns ParkingSpots)
-  = Association (ParkingSpot used by Vehicle)
-  = Inheritance (Car, Bus inherit from Vehicle)

Part 2: SOLID Design Principles (50 Marks)

Question 1: Single Responsibility Principle (12 Marks)

Part A: Identify Responsibilities (6 Marks)

Question: Identify two distinct responsibilities that could be extracted from a monolithic ParkingLot class.

Model Answer: Responsibility 1: Fee Calculation

- Current: ParkingLot calculates parking fees
- Problem: Changes to fee structure require modifying ParkingLot

Responsibility 2: Report Generation

- Current: ParkingLot generates parking reports
- Problem: Changes to report format require modifying ParkingLot

Part B: Proposed Classes (6 Marks)

Question: Propose new classes for each responsibility and explain improvements.

Model Answer:

cpp

// 1. FeeCalculator class

```
class FeeCalculator {
public:
    double calculateFee(Vehicle* vehicle, int hours) {
        return vehicle->calculateParkingFee(hours);
    }

    double applyDiscount(double fee, Vehicle* vehicle) {
        // Discount logic based on vehicle type
        if (dynamic_cast<ElectricVehicle*>(vehicle)) {
            return fee * 0.8; // 20% discount for electric vehicles
        }
        return fee;
    }
};
```

// 2. ReportGenerator class

```
class ReportGenerator {
public:
    void generateDailyReport(const std::vector<Vehicle*>& vehicles) {
        std::cout << "=== Daily Parking Report ===" << std::endl;
        for (const auto* vehicle : vehicles) {
            std::cout << "License: " << vehicle->getLicensePlate() << std::endl;
        }
    }

    void generateMonthlyReport(const std::vector<Vehicle*>& vehicles) {
        std::cout << "=== Monthly Parking Report ===" << std::endl;
        std::cout << "Total vehicles parked: " << vehicles.size() << std::endl;
    }
};
```

Improvements:

- **Better Maintainability:** Each class has single responsibility
 - **Easier Testing:** Can test fee calculation independently
 - **Reduced Coupling:** Changes to reporting don't affect parking logic
 - **Better Reusability:** FeeCalculator can be used by other systems
-

Question 2: Open/Closed Principle (12 Marks)

Question: Design Vehicle and Fee Calculation classes to be open for extension but closed for modification.

Model Answer:**Vehicle Hierarchy Design:**

cpp

```

class Vehicle {
protected:
    std::string licensePlate;
    std::chrono::system_clock::time_point entryTime;

public:
    Vehicle(const std::string& plate) : licensePlate(plate) {}
    virtual ~Vehicle() = default;

    virtual double calculateParkingFee(int hours) const = 0;
    virtual SpotSize getRequiredSpotSize() const = 0;

    std::string getLicensePlate() const { return licensePlate; }
    void setEntryTime(const std::chrono::system_clock::time_point& time) { entryTime = time; }
    std::chrono::system_clock::time_point getEntryTime() const { return entryTime; }
};

```

```

class Car : public Vehicle {
public:
    Car(const std::string& plate) : Vehicle(plate) {}

    double calculateParkingFee(int hours) const override {
        return hours * 2.0;
    }

    SpotSize getRequiredSpotSize() const override {
        return SpotSize::MEDIUM;
    }
};

```

// Easy to add new vehicle types without modifying existing code

```

class ElectricVehicle : public Vehicle {
public:

```

```
ElectricVehicle(const std::string& plate) : Vehicle(plate) {}

double calculateParkingFee(int hours) const override {
    return hours * 1.5; // Discount for electric vehicles
}

SpotSize getRequiredSpotSize() const override {
    return SpotSize::MEDIUM;
}

};
```

Fee Calculation Strategy Design:

cpp

```

class FeeStrategy {
public:
    virtual ~FeeStrategy() = default;
    virtual double calculateFee(Vehicle* vehicle, int hours) = 0;
};

class StandardFeeStrategy : public FeeStrategy {
public:
    double calculateFee(Vehicle* vehicle, int hours) override {
        return vehicle->calculateParkingFee(hours);
    }
};

class SurgePricingStrategy : public FeeStrategy {
public:
    double calculateFee(Vehicle* vehicle, int hours) override {
        double baseFee = vehicle->calculateParkingFee(hours);
        return baseFee * 1.5; // 50% surge pricing
    }
};

class FeeCalculator {
private:
    std::unique_ptr<FeeStrategy> strategy;

public:
    FeeCalculator(std::unique_ptr<FeeStrategy> feeStrategy)
        : strategy(std::move(feeStrategy)) {}

    void setStrategy(std::unique_ptr<FeeStrategy> newStrategy) {
        strategy = std::move(newStrategy);
    }
}

```

```
double calculateFee(Vehicle* vehicle, int hours) {  
    return strategy->calculateFee(vehicle, hours);  
}  
};
```

SOLID Principle Compliance: Open/Closed Principle

- Classes are open for extension (new vehicle types, fee strategies)
 - Classes are closed for modification (existing code unchanged)
-

Question 3: Liskov Substitution Principle (12 Marks)

Question: Describe a scenario where violating LSP could lead to unexpected behavior.

Model Answer:

LSP Violation Scenario:

cpp


```
#include <stdexcept>
```

```
class Vehicle {
```

```
public:
```

```
    virtual ~Vehicle() = default;
```

```
    virtual void park() {
```

```
        // Standard parking procedure
```

```
        std::cout << "Vehicle parked successfully" << std::endl;
```

```
    }
```

```
};
```

```
class Car : public Vehicle {
```

```
public:
```

```
    void park() override {
```

```
        // Car-specific parking (works fine)
```

```
        std::cout << "Car parked successfully" << std::endl;
```

```
    }
```

```
};
```

```
class Motorcycle : public Vehicle {
```

```
public:
```

```
    void park() override {
```

```
        // VIOLATION: Throws exception unexpectedly
```

```
        throw std::runtime_error("Motorcycles need special parking");
```

```
    }
```

```
};
```

```
// Client code breaks when using Motorcycle
```

```
void parkVehicle(Vehicle* vehicle) {
```

```
    vehicle->park(); // Crashes when vehicle is Motorcycle
```

```
}
```

Problems with Violation:

- Client code expecting Vehicle behavior crashes
- Motorcycle cannot be substituted for Vehicle
- Breaks polymorphism expectations

LSP-Compliant Solution:

cpp

```
class Vehicle {
public:
    virtual ~Vehicle() = default;
    virtual bool park() = 0;
    virtual std::string getParkingResult() const = 0;
};

class Car : public Vehicle {
public:
    bool park() override {
        // Car parking logic
        return true;
    }

    std::string getParkingResult() const override {
        return "Car parked successfully";
    }
};

class Motorcycle : public Vehicle {
private:
    bool parkInSpecialArea() {
        // Special parking logic for motorcycles
        return true;
    }

public:
    bool park() override {
        // Motorcycle parking logic (doesn't throw exception)
        return parkInSpecialArea();
    }

    std::string getParkingResult() const override {
```

```

        return "Motorcycle parked in special area";
    }
};

// Client code works with both types
void parkVehicle(Vehicle* vehicle) {
    if (vehicle->park()) {
        std::cout << vehicle->getParkingResult() << std::endl;
    }
}

```

Guidelines for LSP Compliance:

1. **Behavioral Consistency:** Subclasses should behave consistently with superclass
 2. **No Strengthened Preconditions:** Don't add stricter requirements in subclasses
 3. **No Weakened Postconditions:** Don't provide weaker guarantees in subclasses
 4. **Exception Compatibility:** Don't throw new exceptions in subclass methods
-

Question 4: Interface Segregation Principle (14 Marks)

Question: Apply ISP to design granular interfaces instead of monolithic ParkingLotInterface.

Model Answer:

Problem - Monolithic Interface:

cpp

```
class ParkingLotInterface {  
public:  
    virtual ~ParkingLotInterface() = default;  
    virtual void parkVehicle(Vehicle* vehicle) = 0;  
    virtual void unparkVehicle(const std::string& licensePlate) = 0;  
    virtual void processPayment(double amount) = 0;  
    virtual void generateReport() = 0;  
    virtual void updateDisplayBoard(const std::string& message) = 0;  
    virtual void activateSecurityCamera(int cameraId) = 0;  
    virtual void sendNotification(const std::string& message) = 0;  
};
```

Solution - Segregated Interfaces:

cpp

// 1. Parking Operations Interface

```
class ParkingOperations {  
public:  
    virtual ~ParkingOperations() = default;  
    virtual bool parkVehicle(Vehicle* vehicle) = 0;  
    virtual bool unparkVehicle(const std::string& licensePlate) = 0;  
    virtual bool isSpotAvailable(SpotSize size) = 0;  
};
```

// 2. Payment Operations Interface

```
class PaymentOperations {  
public:  
    virtual ~PaymentOperations() = default;  
    virtual bool processPayment(double amount) = 0;  
    virtual void generateReceipt(const std::string& licensePlate) = 0;  
};
```

// 3. Display Operations Interface

```
class DisplayOperations {  
public:  
    virtual ~DisplayOperations() = default;  
    virtual void updateDisplayBoard(const std::string& message) = 0;  
    virtual void showAvailableSpots(int count) = 0;  
};
```

// 4. Security Operations Interface

```
class SecurityOperations {  
public:  
    virtual ~SecurityOperations() = default;  
    virtual void activateSecurityCamera(int cameraId) = 0;  
    virtual void logSecurityEvent(const std::string& event) = 0;  
};
```

```
// 5. Reporting Operations Interface  
class ReportingOperations {  
public:  
    virtual ~ReportingOperations() = default;  
    virtual void generateDailyReport() = 0;  
    virtual void generateMonthlyReport() = 0;  
};
```

Implementation Examples:

cpp

```

class ParkingLot : public ParkingOperations {
private:
    std::vector<std::unique_ptr<ParkingSpot>> spots;
    std::vector<Vehicle*> parkedVehicles;

public:
    bool parkVehicle(Vehicle* vehicle) override {
        // Only implements parking-related methods
        SpotSize requiredSize = vehicle->getRequiredSpotSize();

        for (auto& spot : spots) {
            if (spot->isAvailable() && spot->getSize() == requiredSize) {
                if (spot->assignVehicle(vehicle)) {
                    parkedVehicles.push_back(vehicle);
                    return true;
                }
            }
        }
        return false;
    }

    bool unparkVehicle(const std::string& licensePlate) override {
        // Find and remove vehicle
        for (auto it = parkedVehicles.begin(); it != parkedVehicles.end(); ++it) {
            if ((*it)->getLicensePlate() == licensePlate) {
                parkedVehicles.erase(it);
                return true;
            }
        }
        return false;
    }

    bool isSpotAvailable(SpotSize size) override {

```



```

    for (const auto& spot : spots) {
        if (spot->isAvailable() && spot->getSize() == size) {
            return true;
        }
    }
    return false;
}
};

```

```

class PaymentGateway : public PaymentOperations {
public:
    bool processPayment(double amount) override {
        // Payment processing logic
        std::cout << "Processing payment of $" << amount << std::endl;
        return true;
    }

    void generateReceipt(const std::string& licensePlate) override {
        std::cout << "Receipt generated for vehicle: " << licensePlate << std::endl;
    }
};

```

```

class DisplayBoard : public DisplayOperations {
public:
    void updateDisplayBoard(const std::string& message) override {
        std::cout << "Display: " << message << std::endl;
    }

    void showAvailableSpots(int count) override {
        std::cout << "Available spots: " << count << std::endl;
    }
};

```

```

class SecuritySystem : public SecurityOperations {
public:
    void activateSecurityCamera(int cameraId) override {
        std::cout << "Camera " << cameraId << " activated" << std::endl;
    }

    void logSecurityEvent(const std::string& event) override {
        std::cout << "Security log: " << event << std::endl;
    }
};

class ReportSystem : public ReportingOperations {
public:
    void generateDailyReport() override {
        std::cout << "Generating daily report..." << std::endl;
    }

    void generateMonthlyReport() override {
        std::cout << "Generating monthly report..." << std::endl;
    }
};

```

Benefits of Interface Segregation:

- **Focused Interfaces:** Each interface has a single, well-defined purpose
- **Reduced Dependencies:** Classes only implement interfaces they need
- **Better Testability:** Easy to mock specific interfaces
- **Improved Maintainability:** Changes to one interface don't affect others
- **Enhanced Flexibility:** Different classes can implement different combinations of interfaces

Additional C++ Considerations for SOLID Principles

Memory Management and RAII

cpp

```
class ParkingLotManager {
private:
    std::unique_ptr<ParkingOperations> parkingOps;
    std::unique_ptr<PaymentOperations> paymentOps;
    std::unique_ptr<ReportingOperations> reportingOps;

public:
    ParkingLotManager() {
        parkingOps = std::make_unique<ParkingLot>();
        paymentOps = std::make_unique<PaymentGateway>();
        reportingOps = std::make_unique<ReportSystem>();
    }

    // Automatic cleanup due to RAII
    ~ParkingLotManager() = default;
};
```

Template-based Strategy Pattern

cpp

```
template<typename FeeStrategy>
class TemplateFeeCalculator {
private:
    FeeStrategy strategy;

public:
    double calculateFee(Vehicle* vehicle, int hours) {
        return strategy.calculateFee(vehicle, hours);
    }
};

// Usage
TemplateFeeCalculator<StandardFeeStrategy> standardCalculator;
TemplateFeeCalculator<SurgePricingStrategy> surgeCalculator;
```

This completes the comprehensive C++ study guide with practical assignment questions and answers that demonstrate proper application of OOP principles and SOLID design patterns.